

TP : Détection de voies routières

Vision par ordinateur avec OpenCV en C++

Maryam Boufous

1. Objectifs du TP

L'objectif de ce TP est d'implémenter un système complet de détection de voies routières en utilisant les techniques de vision par ordinateur avec OpenCV en C++. Vous implémenterez étape par étape les différentes phases du traitement d'image.

Compétences évaluées :

- Compréhension des espaces de couleur HSV
- Transformation de perspective (*bird eye view*)
- Détection de contours et seuillage (algorithme de Canny)
- Méthode des fenêtres glissantes (*sliding windows*)
- Ajustement polynomial et calcul de courbure

2. Vue d'ensemble du pipeline

Le système de détection de voies suit un pipeline séquentiel de 10 étapes. Chaque étape transforme l'image pour extraire progressivement l'information sur les voies.

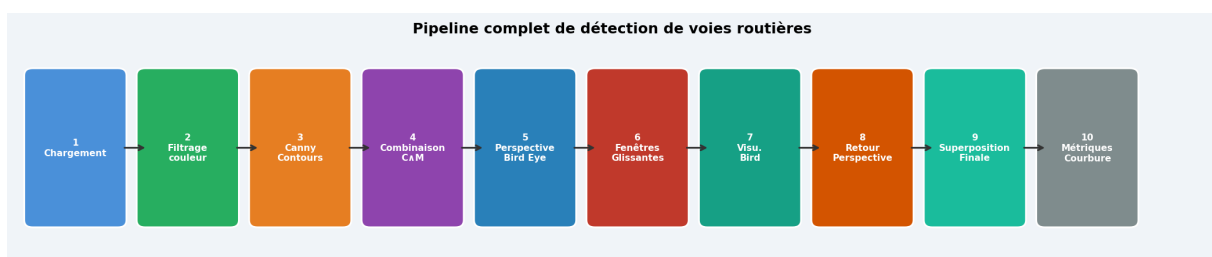


Figure 1: Vue d'ensemble du pipeline complet de détection de voies routières

3. Concepts fondamentaux et définitions

3.1 Espace de couleur HSV

Définition — Espace HSV (Hue, Saturation, Value)

L'espace HSV décompose une couleur en trois composantes indépendantes :

- **H (Teinte / Hue)** : la couleur pure, représentée par un angle. Dans OpenCV, la plage est **0–180** (au lieu de 0–360° en théorie) afin de tenir sur un octet.
- **S (Saturation)** : la pureté ou intensité de la couleur, de 0 (gris pur) à 255 (couleur saturée). Une saturation faible correspond à du blanc ou du gris.
- **V (Valeur / Luminosité)** : la brillance, de 0 (noir) à 255 (très lumineux).

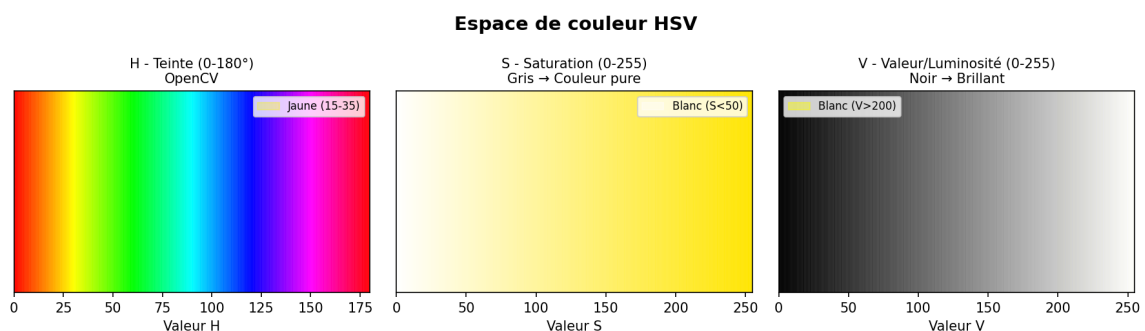


Figure 2: Les trois composantes HSV et les plages utilisées pour détecter le blanc et le jaune

Pourquoi HSV plutôt que RGB ?

En espace RGB, les variations d'éclairage (ombre, soleil, nuit) modifient *simultanément* les trois canaux R, G et B, rendant difficile la définition d'un seuil de couleur stable. En HSV, la teinte H reste constante quelle que soit la luminosité : seul V change. On peut donc détecter le **jaune** (H entre 15 et 35) ou le **blanc** ($S < 50$, $V > 200$) de façon robuste.

Valeurs HSV typiques pour les marquages routiers :

Couleur	H (Teinte)	S (Saturation)	V (Valeur)
Blanc	0 – 180	0 – 50	200 – 255
Jaune	15 – 35	60 – 255	100 – 255

3.2 Algorithme de Canny — Détection de contours

Définition — Algorithme de Canny (1986)

L'algorithme de Canny détecte les **gradients d'intensité** dans l'image, c'est-à-dire les zones où la luminosité change brusquement (= les bords des objets). Il se compose de 4 étapes internes :

1. **Lissage gaussien** : suppression du bruit par convolution avec un noyau gaussien.
2. **Calcul du gradient** : détection de la magnitude et direction du gradient en chaque pixel (filtres de Sobel).
3. **Suppression des non-maxima** : on ne conserve que les pixels qui sont des maxima locaux dans la direction du gradient (amincissement des contours).
4. **Double seuillage par hystérésis** :
 - Pixel avec gradient $>$ seuil haut : **contour certain** (conservé).
 - Pixel avec gradient $<$ seuil bas : **rejeté** (bruit).
 - Pixel entre les deux seuils : conservé **seulement** s'il est connecté à un contour certain.

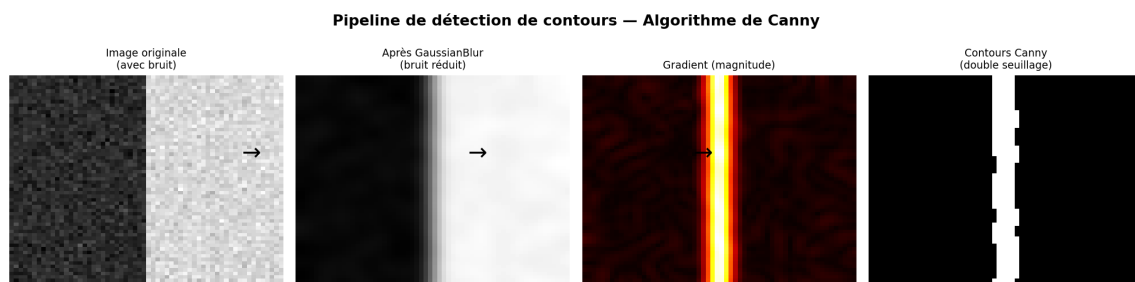


Figure 3: Pipeline interne de l'algorithme de Canny : du bruitage aux contours nets

Choix des seuils Canny

Les seuils 50 et 150 sont des valeurs de départ courantes. Un rapport 1:3 est recommandé. Si les seuils sont trop bas, du bruit est détecté ; trop hauts, des contours réels sont perdus.

3.3 Transformation de perspective

Définition — Transformation homographique (Perspective Transform)

Une **transformation de perspective** (ou homographie) mappe 4 points sources vers 4 points destination dans un plan différent. Elle modifie l'angle de vue apparent de la scène :

- Vue caméra inclinée : les voies convergent vers un point de fuite (effet perspective).
- Vue de dessus (*bird eye view*) : les voies sont parallèles et la distance est proportionnelle à la position verticale dans l'image.

Elle est représentée par une matrice 3×3 et s'applique aux coordonnées homogènes.

Transformation de perspective — Vue caméra → Vue de dessus (Bird Eye View)

Vue caméra (perspective)
Lignes convergentes

Vue de dessus (bird eye)
Lignes parallèles



Figure 4: Passage de la vue caméra (trapèze rouge) à la vue bird eye (rectangle bleu)

Différence transformation affine vs perspective :

- **Affine** (matrice 2×3) : préserve les lignes parallèles. Utilisée pour rotation, translation, mise à l'échelle.
- **Perspective** (matrice 3×3) : ne préserve *pas* le parallélisme. Utilisée pour simuler un changement de point de vue : c'est notre cas ici.

3.4 Méthode des fenêtres glissantes

Définition — Sliding Windows — Fenêtres glissantes

La méthode des fenêtres glissantes divise l'image verticalement en N bandes horizontales. Pour chaque bande (du bas vers le haut) :

1. Une fenêtre rectangulaire est placée autour de la position courante de la voie.
2. On collecte tous les pixels blancs à l'intérieur de cette fenêtre.
3. Si le nombre de pixels collectés dépasse un seuil (`min_pixels`), on recentre la prochaine fenêtre sur la moyenne de leurs positions en x .

La position de départ (en bas) est déterminée par les pics de l'**histogramme** de la moitié basse de l'image.

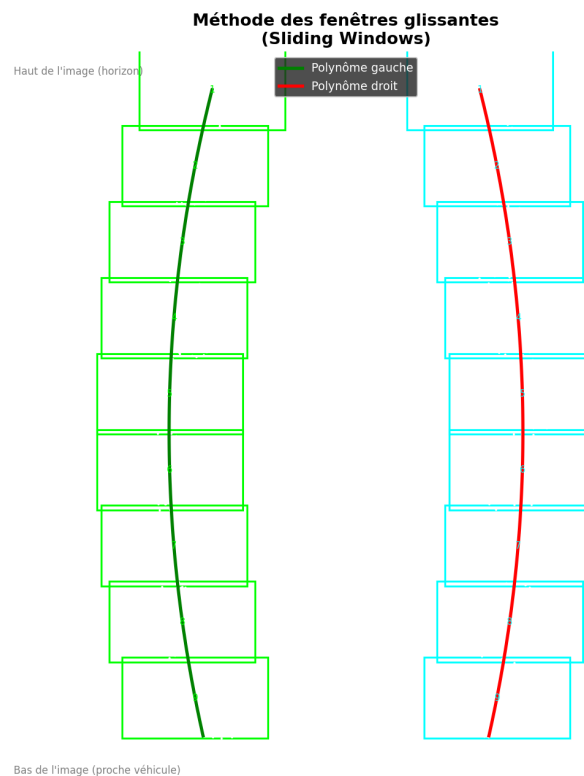


Figure 5: Fenêtres glissantes (vert = voie gauche, cyan = voie droite) et polynômes ajustés

3.5 Ajustement polynomial et rayon de courbure

Définition — Ajustement polynomial — Moindres carrés

On modélise chaque voie par un polynôme du second degré :

$$x = Ay^2 + By + C$$

où x est la position horizontale et y la position verticale dans l'image (on ajuste x en fonction de y et non l'inverse, car les voies peuvent être presque verticales).

Les coefficients A , B , C sont trouvés par la **méthode des moindres carrés** : on résout le système sur-déterminé $\mathbf{A} \mathbf{c} = \mathbf{b}$ (via décomposition SVD dans OpenCV avec `solve(..., DECOMP_SVD)`).

Définition — Rayon de courbure

Pour une courbe $x = f(y)$, le rayon de courbure en un point est :

$$R = \frac{\left[1 + \left(\frac{dx}{dy} \right)^2 \right]^{3/2}}{\left| \frac{d^2x}{dy^2} \right|}$$

Pour notre parabole $x = Ay^2 + By + C$: $\frac{dx}{dy} = 2Ay + B$ et $\frac{d^2x}{dy^2} = 2A$, d'où :

$$R = \frac{[1 + (2Ay + B)^2]^{3/2}}{|2A|}$$

Un grand R indique une route presque droite ; un petit R un virage serré.

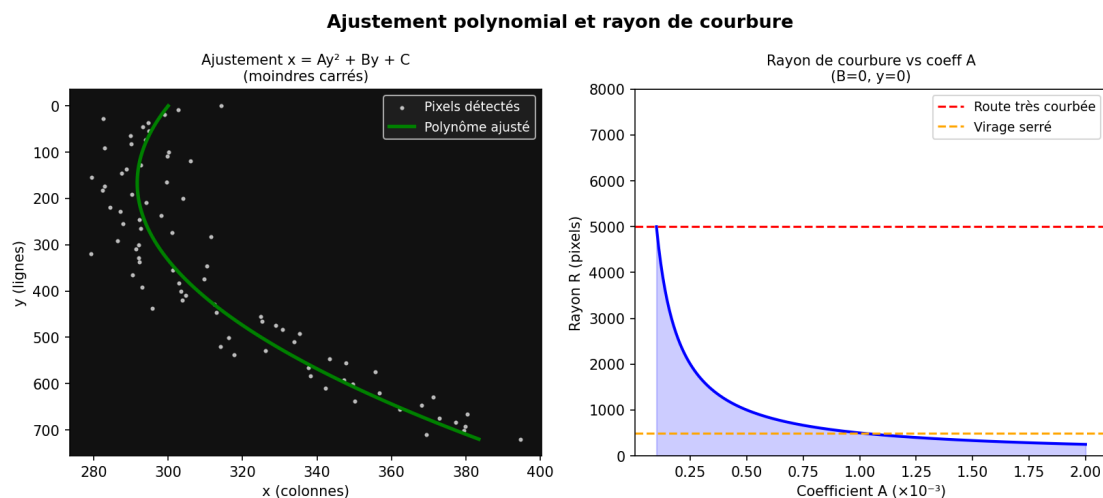


Figure 6: Ajustement polynomial sur les pixels de voie (gauche) et variation du rayon de courbure (droite)

4. Fonctions OpenCV essentielles

Fonction	Utilisation
<code>cvtColor()</code>	Conversion d'espaces : BGR $\rightarrow$ HSV, RGB, GRAY
<code>inRange()</code>	Masquage binaire par seuils HSV
<code>GaussianBlur()</code>	Réduction du bruit avant Canny
<code>Canny()</code>	Détection de contours par double seuillage
<code>bitwise_or()</code> / <code>bitwise_and()</code>	Combinaison logique de masques binaires
<code>getPerspectiveTransform()</code>	Calcul de la matrice de transformation 3×3
<code>warpPerspective()</code>	Application de la transformation de perspective
<code>findNonZero()</code>	Extraction de la liste de pixels non nuls
<code>reduce()</code>	Calcul d'histogramme (somme par colonne)
<code>minMaxLoc()</code>	Localisation du pic dans l'histogramme
<code>solve()</code>	Résolution par moindres carrés (SVD) pour le polynôme
<code>polyLines()</code>	Dessin des lignes de voie détectées
<code>fillPoly()</code>	Remplissage de la zone entre les voies
<code>addWeighted()</code>	Superposition semi-transparente sur l'image originale
<code>putText()</code>	Affichage des métriques (rayon, décalage)

5. Étapes détaillées du pipeline

5.1 Étape 1 : Chargement et prétraitement

Principe

Charger l'image depuis le disque et la convertir dans un format adapté au traitement.

Rôle

Point d'entrée du pipeline ; prépare l'image pour les traitements ultérieurs. OpenCV charge en BGR par défaut ; on convertit en RGB pour l'affichage final correct.

```

1 Mat image = imread("route.jpg");
2 if (image.empty()) {
3     cerr << "Erreur: Impossible de charger l'image" << endl;
4     return -1;
5 }
6 Mat image_rgb;
7 cvtColor(image, image_rgb, COLOR_BGR2RGB);

```

Listing 1: Chargement de l'image

5.2 Étape 2 : Filtrage par couleur

Principe

Les marquages routiers sont généralement blancs ou jaunes. L'espace HSV permet d'isoler ces couleurs indépendamment des conditions d'éclairage, contrairement à l'espace RGB qui est sensible aux variations de luminosité.

Rôle

Créer un masque binaire qui ne garde que les pixels correspondant aux couleurs des voies.

```
1 Mat hsv;
2 cvtColor(image, hsv, COLOR_BGR2HSV);
3
4 // Masque pour le blanc : saturation faible + luminosité élevée
5 Scalar blanc_min(0, 0, 200);
6 Scalar blanc_max(180, 50, 255);
7 Mat masque_blanc;
8 inRange(hsv, blanc_min, blanc_max, masque_blanc);
9
10 // Masque pour le jaune : teinte 15-35, saturation et luminosité
    élevées
11 Scalar jaune_min(15, 60, 100);
12 Scalar jaune_max(35, 255, 255);
13 Mat masque_jaune;
14 inRange(hsv, jaune_min, jaune_max, masque_jaune);
15
16 // Combinaison : conserver les pixels blancs OU jaunes
17 Mat masque_couleur;
18 bitwise_or(masque_blanc, masque_jaune, masque_couleur);
```

Listing 2: Filtrage couleur en HSV

5.3 Étape 3 : Détection de contours avec Canny

Principe

L'algorithme de Canny détecte les forts gradients d'intensité : la luminosité change brusquement aux bordures des marquages routiers. Le flou gaussien préalable évite de détecter le bruit pixel par pixel.

Rôle

Identifier les contours présents dans l'image, incluant ceux des marquages routiers.

```
1 Mat gris;
2 cvtColor(image, gris, COLOR_BGR2GRAY);
3
4 Mat flou;
5 GaussianBlur(gris, flou, Size(5, 5), 0); // Réduction du bruit
6
7 // Seuil bas = 50, seuil haut = 150 (rapport 1:3 recommande)
```



```
8 Mat contours;  
9 Canny(flou, contours, 50, 150);
```

Listing 3: Détection de contours Canny

5.4 Étape 4 : Combinaison contours + couleur

Principe

Intersection logique (AND) entre les contours détectés et le masque de couleur. Cette opération ne conserve que les pixels qui répondent aux deux critères simultanément.

Rôle

Éliminer les contours parasites (arbres, bâtiments, véhicules...) en ne gardant que ceux qui ont la bonne couleur.

```
1 // Seuls les contours dans les zones blanc/jaune sont conservés  
2 Mat combine;  
3 bitwise_and(contours, contours, combine, masque_couleur);
```

Listing 4: Combinaison des masques

5.5 Étape 5 : Transformation de perspective (vue bird)

Principe

La route vue de face a un effet de perspective : les lignes convergent vers l'horizon. En appliquant une transformation homographique, on obtient une vue de dessus (*bird eye view*) où les lignes sont parallèles.

Rôle

Éliminer l'effet de perspective pour que les voies soient parallèles et que la distance soit proportionnelle à la position verticale.

```
1 // Points source : trapeze dans l'image originale  
2 vector<Point2f> pts_source = {  
3     Point2f(largeur * 0.15f, hauteur),           // Bas gauche  
4     Point2f(largeur * 0.45f, hauteur * 0.60f),  // Haut gauche  
5     Point2f(largeur * 0.55f, hauteur * 0.60f),  // Haut droite  
6     Point2f(largeur * 0.85f, hauteur)           // Bas droite  
7 };  
8  
9 // Points destination : rectangle en vue bird  
10 vector<Point2f> pts_destination = {  
11     Point2f(0, hauteur), // Bas gauche  
12     Point2f(0, 0),       // Haut gauche  
13     Point2f(largeur, 0),  // Haut droite  
14     Point2f(largeur, hauteur) // Bas droite  
15 };  
16
```

```

17 // Calcul des matrices (directe et inverse)
18 Mat M      = getPerspectiveTransform(pts_source, pts_destination);
19 Mat M_inv  = getPerspectiveTransform(pts_destination, pts_source);
20
21 Mat vue_bird;
22 warpPerspective(combine, vue_bird, M, Size(largeur, hauteur));

```

Listing 5: Transformation de perspective

5.6 Étape 6 : Détection des voies par fenêtres glissantes

6.1 — Calcul d'histogramme

Principe En sommant les pixels blancs verticalement dans la moitié basse de l'image, on obtient un histogramme dont les pics indiquent les positions horizontales des voies.

Rôle Trouver le point de départ de chaque voie en bas de l'image.

```

1 Mat moitie_basse = image_bird(Rect(0, image_bird.rows / 2,
2                                image_bird.cols, image_bird.
3                                rows / 2));
4 Mat histogramme;
5 reduce(moitie_basse, histogramme, 0, REDUCE_SUM, CV_32S);
6
7 int milieu = histogramme.cols / 2;
8 Point pic_gauche, pic_droite;
9
10 minMaxLoc(histogramme(Rect(0, 0, milieu, 1)),
11            nullptr, nullptr, nullptr, &pic_gauche);
12 minMaxLoc(histogramme(Rect(milieu, 0, histogramme.cols - milieu,
13                             1)),
14            nullptr, nullptr, nullptr, &pic_droite);
15
16 int x_base_gauche = pic_gauche.x;
17 int x_base_droite = pic_droite.x + milieu;

```

Listing 6: Calcul d'histogramme et localisation des bases

6.2 — Fenêtres glissantes

Principe Les voies sont continues verticalement. En remontant avec des fenêtres qui se recentrent sur la moyenne des pixels détectés, on capture l'ensemble des pixels de chaque voie.

Rôle Extraire tous les pixels appartenant à chaque voie.

```

1 int nb_fenetres      = 9;
2 int hauteur_fenetre  = image_bird.rows / nb_fenetres;
3 int marge            = 100;    // +/- pixels autour de la position
4                               // courante
5 int min_pixels       = 50;    // seuil de recentrage

```

```

6  int x_courant_gauche = x_base_gauche;
7  int x_courant_droite = x_base_droite;
8
9  vector<Point> pixels_voie_gauche, pixels_voie_droite;
10
11 for (int fenetre = 0; fenetre < nb_fenetres; fenetre++) {
12     int y_bas = image_bird.rows - (fenetre + 1) *
        hauteur_fenetre;
13     int y_haut = image_bird.rows - fenetre *
        hauteur_fenetre;
14
15     vector<Point> bons_gauche, bons_droite;
16
17     for (const Point& p : points_non_nuls) {
18         if (p.y >= y_bas && p.y < y_haut) {
19             if (p.x >= x_courant_gauche - marge && p.x <
                x_courant_gauche + marge)
20                 bons_gauche.push_back(p);
21             if (p.x >= x_courant_droite - marge && p.x <
                x_courant_droite + marge)
22                 bons_droite.push_back(p);
23         }
24     }
25
26     pixels_voie_gauche.insert(pixels_voie_gauche.end(),
27                               bons_gauche.begin(), bons_gauche.
28                               end());
29     pixels_voie_droite.insert(pixels_voie_droite.end(),
30                               bons_droite.begin(), bons_droite.
31                               end());
32
33     // Recentrage si assez de pixels trouves
34     if (bons_gauche.size() > (size_t)min_pixels) {
35         int somme = 0;
36         for (const Point& p : bons_gauche) somme += p.x;
37         x_courant_gauche = somme / (int)bons_gauche.size();
38     }
39     if (bons_droite.size() > (size_t)min_pixels) {
40         int somme = 0;
41         for (const Point& p : bons_droite) somme += p.x;
42         x_courant_droite = somme / (int)bons_droite.size();
43     }
44 }

```

Listing 7: Fenêtres glissantes

6.3 — Ajustement polynomial (moindres carrés)

Principe En vue bird, les voies suivent une parabole due aux courbes de la route. On résout par SVD le système $\mathbf{A}\mathbf{c} = \mathbf{b}$ pour trouver les coefficients $[A, B, C]$ de $x = Ay^2 + By + C$.

Rôle Modéliser mathématiquement chaque voie par une équation du second degré.

```

1 Mat A_gauche(points_gauche.size(), 3, CV_32F);
2 Mat b_gauche(points_gauche.size(), 1, CV_32F);
3
4 for (size_t i = 0; i < points_gauche.size(); i++) {
5     float y = points_gauche[i].x;          // y est la 1ere
6         coordonnee ici
7     A_gauche.at<float>(i, 0) = y * y;      // y^2
8     A_gauche.at<float>(i, 1) = y;         // y
9     A_gauche.at<float>(i, 2) = 1.0f;      // constante
10    b_gauche.at<float>(i, 0) = points_gauche[i].y; // x cible
11 }
12 Mat coeffs_gauche;
13 solve(A_gauche, b_gauche, coeffs_gauche, DECOMP_SVD);
14 // coeffs_gauche = [A, B, C]

```

Listing 8: Ajustement polynomial par moindres carrés

6.4 — Génération des points de la courbe

Principe Une fois les coefficients connus, on calcule x pour chaque y sur toute la hauteur de l'image.

Rôle Créer une ligne continue et lisse représentant chaque voie.

```

1 for (int y = 0; y < image_bird.rows; y++) {
2     float x_gauche = coeffs_gauche[0] * y * y
3         + coeffs_gauche[1] * y
4         + coeffs_gauche[2];
5     points_gauche.push_back(Point2f(x_gauche, y));
6 }

```

Listing 9: Génération des points de voie

5.7 Étape 7 : Visualisation en vue bird

Principe

Dessiner les voies détectées sur une image noire dans l'espace transformé, puis remplir la zone entre les deux voies.

Rôle

Vérifier visuellement la qualité de l'ajustement polynomial avant de reprojeter.

```

1 Mat voies_bird = Mat::zeros(vue_bird.size(), CV_8UC3);
2
3 vector<Point> pts_gauche_int, pts_droite_int;
4 for (const Point2f& p : voies.points_gauche)
5     pts_gauche_int.push_back(Point(cvRound(p.x), cvRound(p.y)));
6 for (const Point2f& p : voies.points_droite)

```

```

7     pts_droite_int.push_back(Point(cvRound(p.x), cvRound(p.y)));
8
9     polylines(voies_bird, pts_gauche_int, false, Scalar(0, 255, 0),
10             25);
11
12     polylines(voies_bird, pts_droite_int, false, Scalar(0, 255, 0),
13             25);
14
15     // Remplissage de la zone entre les voies
16     vector<Point> zone;
17     zone.insert(zone.end(), pts_gauche_int.begin(), pts_gauche_int.
18         end());
19     zone.insert(zone.end(), pts_droite_int.rbegin(), pts_droite_int.
20         rend());
21     vector<vector<Point>> zone_a_remplir = {zone};
22     fillPoly(voies_bird, zone_a_remplir, Scalar(0, 255, 0));

```

Listing 10: Visualisation en vue bird

5.8 Étape 8 : Retour à la perspective originale

Principe

Appliquer la transformation inverse (matrice M^{-1}) pour revenir de la vue bird à la perspective normale de la caméra.

Rôle

Reporter les voies détectées dans l'espace de l'image originale.

```

1 Mat voies_original;
2 warpPerspective(voies_bird, voies_original, M_inv, Size(largeur,
    hauteur));

```

Listing 11: Retour à la perspective originale

5.9 Étape 9 : Superposition sur l'image originale

Principe

Combinaison linéaire de l'image originale et du masque des voies :

$$\text{résultat} = \alpha \cdot \text{image} + \beta \cdot \text{voies} + \gamma$$

avec $\alpha = 1.0$, $\beta = 0.5$, $\gamma = 0$.

Rôle

Afficher les voies détectées par-dessus la scène routière réelle pour une visualisation intuitive et semi-transparente.

```

1 Mat resultat;
2 addWeighted(image_rgb, 1.0, voies_original, 0.5, 0, resultat);

```

Listing 12: Superposition semi-transparente

5.10 Étape 10 : Calcul des métriques

10.1 — Rayon de courbure

Principe Les coefficients du polynôme sont en pixels. On les convertit d'abord en mètres à l'aide de facteurs d'échelle, puis on applique la formule du rayon de courbure.

```

1 // Facteurs de conversion pixels -> metres
2 double m_par_pixel_y = 30.0 / 720.0;    // 30 m de route sur 720
    pixels
3 double m_par_pixel_x = 3.7 / 700.0;    // largeur voie standard
    3.7 m
4
5 // Conversion des coefficients en metres
6 double A_m = coeffs[0] * m_par_pixel_x / (m_par_pixel_y *
    m_par_pixel_y);
7 double B_m = coeffs[1] * m_par_pixel_x / m_par_pixel_y;
8
9 double y_eval = (vue_bird.rows - 1) * m_par_pixel_y; // bas de
    l'image
10 double courbure = pow(1.0 + pow(2.0 * A_m * y_eval + B_m, 2),
    1.5)
11
    / fabs(2.0 * A_m);

```

Listing 13: Calcul du rayon de courbure

10.2 — Décalage latéral du véhicule

Principe Le centre de la voie est la moyenne des positions gauche et droite en bas de l'image. La caméra (centre de l'image) représente le centre du véhicule. La différence convertie en mètres donne le décalage latéral.

```

1 double centre_voie = (voies.points_gauche.back().x
2                     + voies.points_droite.back().x) / 2.0;
3 double centre_vehicule = vue_bird.cols / 2.0;
4 double decalage_pixels = centre_vehicule - centre_voie;
5 double decalage_metres = decalage_pixels * m_par_pixel_x;
6
7 putText(resultat,
8         "Decalage: " + to_string(fabs(decalage_metres)).substr(0,
9         4)
10        + " m " + (decalage_metres > 0 ? "a gauche" : "a droite")
        ,
        Point(50, 120), FONT_HERSHEY_SIMPLEX, 1.2, Scalar(255,
        255, 255), 3);

```

Listing 14: Calcul du décalage latéral

6. Travail à réaliser

6.1 Partie 1 : Implémentation de base

1. Téléchargez le code fourni et une image de test routière.
2. Compilez et exécutez le programme pour vérifier son fonctionnement.
3. Commentez chaque section du code pour expliquer son rôle.
4. Testez avec différentes images (route droite, courbe, nuit, etc.) et notez les résultats.

6.2 Partie 2 : Analyse et optimisation

1. **Ajustement des seuils Canny** : Modifiez les seuils (50 et 150) et observez l'impact.
2. **Plages de couleur HSV** : Les valeurs pour le blanc et le jaune sont-elles optimales ? Proposez des ajustements pour des conditions nocturnes.
3. **Paramètres des fenêtres glissantes** : Faites varier `nb_fenetres`, `marge`, et `min_pixels`. Analysez les résultats.
4. **Points source de la perspective** : Comment les choisir ? Que se passe-t-il s'ils sont mal positionnés ?

6.3 Partie 3 : Améliorations

1. **Voie unique** : Modifiez le code pour fonctionner si une seule voie est visible.
2. **Tracking temporel** : Proposez une méthode pour utiliser les frames précédentes afin de stabiliser la détection.
3. **Interface graphique** : Ajoutez des sliders pour ajuster les paramètres en temps réel avec `createTrackbar()`.
4. **Robustesse** : Comment améliorer la robustesse aux ombres, marquages usés, ou conditions météorologiques ?

7. Questions théoriques

1. Pourquoi utilise-t-on l'espace HSV plutôt que RGB pour le filtrage couleur ?
2. Expliquez le principe de l'algorithme de Canny. Quel est le rôle du double seuillage ?
3. Quelle est la différence entre une transformation affine et une transformation perspective ? Dans quel cas utilise-t-on chacune ?
4. Pourquoi modélise-t-on les voies par un polynôme du second degré plutôt qu'une ligne droite ?
5. Expliquez la méthode des moindres carrés pour l'ajustement polynomial. Pourquoi utilise-t-on la décomposition SVD ?
6. Comment convertit-on des pixels en mètres pour le calcul du rayon de courbure ? Quelles hypothèses fait-on ?
7. Quelles sont les limites de cette approche ? Dans quels cas risque-t-elle d'échouer ?

8. Annexes

8.1 Rappel sur l'espace HSV

Composante	Description
H (Teinte)	Couleur pure. Plage 0–180 dans OpenCV (convention demi-angle).
S (Saturation)	Pureté de la couleur, 0–255. Proche de 0 = gris/blanc.
V (Valeur)	Luminosité, 0–255. Proche de 0 = noir.

Avantage : la teinte H est quasi-invariante aux changements d'éclairage, contrairement à RGB où les trois canaux varient simultanément.

8.2 Formule complète du rayon de courbure

Pour $x = f(y)$:

$$R(y) = \frac{\left(1 + \left[\frac{dx}{dy}\right]^2\right)^{3/2}}{\left|\frac{d^2x}{dy^2}\right|}$$

Pour $x = Ay^2 + By + C$, $\frac{dx}{dy} = 2Ay + B$ et $\frac{d^2x}{dy^2} = 2A$:

$$R(y) = \frac{[1 + (2Ay + B)^2]^{3/2}}{|2A|}$$

On évalue cette formule au bas de l'image (y maximal), là où le véhicule se trouve, après conversion des coefficients en mètres.

8.3 Commandes de compilation

```

1 # Compilation
2 g++ -o detection_voies detection_voies.cpp \
3     'pkg-config --cflags --libs opencv4'
4
5 # Execution
6 ./detection_voies

```

Listing 15: Compilation et exécution avec OpenCV 4